Functions

Like most of the programming languages, a function in PHP is a block of organized, reusable code that is used to perform a single, related action. Functions provide better modularity for your application and a high degree of code reuse.

PHP supports a structured programming approach by arranging the processing logic by defining blocks of independent reusable functions. The main advantage of this approach is that the code becomes easy to follow, develop and maintain.

Types of Functions

You already have seen many functions like **fopen()** and **fread()** etc. They are built-in functions but PHP gives you option to create your own functions as well. There are two types of functions in PHP

- **Built-in functions** PHP's standard library contains a large number of built-in functions for string processing, file IO, mathematical computations and more.
- **User-defined functions** You can create user-defined functions too, specific to the requirements of the programming logic.

A function may be invoked from any other function by passing required data (called **parameters** or **arguments**). The called function returns its result back to the calling environment.

There are two parts which should be clear to you

- Creating a PHP Function
- Calling a PHP Function

In fact you hardly need to create your own PHP function because there are already more than 1000 built-in library functions created for different area and you just need to call them according to your requirement.

Why Use Functions?

Here are some of the reasons provided to explain why you should use functions

- Reusable code is defined as code that can be written once and used repeatedly.
- Breaking code into smaller parts makes it easier to maintain.
- Functions clarify and arrange the code.

Creating a Function in PHP

Now let's understand the process in detail. The first step is to write a function and then you can call it as many times as required.

To create a new **function**, use the function keyword, followed by the name of the function you may want to use. In front of the name, put a parenthesis, which may or may not contain arguments.

It is followed by a block of statements delimited by curly brackets. This function block contains the statements to be executed every time the function is called. The general **syntax** of defining a function is as follows

```
<?php
   function foo($arg_1, $arg_2, $arg_n) {
      statements;
      return $retval;
   }
?>
```

If the function is intended to return some result back to the calling environment, there should be a **return** statement as the last statement in the function block. It is not mandatory to have a **return** statement, as even without it, the program flow goes back to the caller, albeit without carrying any value with it.

Any valid PHP code may appear inside a function, even other functions and class definitions. Name of the function must follow the same rules as used to form the name of a variable. It should start with a letter or underscore, followed by any number of letters, numbers, or underscores.

Here is a simple function in PHP. Whenever called, it is expected to display the message "Hello World".

```
function sayhello() {
   echo "Hello World";
}
```

User-defined Functions in PHP

Its very easy to create your own PHP function. Let's start with a simple example after which we will elaborate how it works. Suppose you want to create a PHP function which will simply write a simple message on your browser when you will call it.

Example

In this example, we create a function called writeMessage() and then call it to print a simple message

```
<?php
   /* Defining a PHP Function */
   function writeMessage() {
```

```
      echo "You are really a nice person, Have a nice time!";
   }

   /* Calling a PHP Function */
   writeMessage();

?>
```

Output

It will produce the following output

```
You are really a nice person, Have a nice time!
```

Calling a Function in PHP

Once a function is defined, it can be called any number of times, from anywhere in the PHP code. Note that a function will not be called automatically.

To call the function, use its name in a statement; the name of the function followed by a semicolon.

```
<?php
   # define a function
   function sayhello(){
      echo "Hello World";
   }
   # calling the function
   sayhello();
?>
```

Output

It will produce the following output

```
Hello World
```

Assuming that the above script "hello.php" is present in the document root folder of the PHP server, open the browser and enter the URL as **http://localhost/hello.php**. You should see the "Hello World" message in the browser window.

In this example, the function is defined without any arguments or any return value. In the subsequent chapters, we shall learn about how to define and pass arguments, and how to make a function return some value. Also, some advanced features of PHP functions such as recursive functions, calling a function by value vs by reference, etc. will also be explained in detail.

Additional Key Concepts in PHP Functions

Here are the key concepts used in php functions

- **Function Parameters and Arguments:** Pass data to functions and specify default values.
- **Return Values:** Use return to send the results back to the caller.
- **Variable Scope:** Understand local and global variables, as well as the global keyword.
- **Anonymous Functions:** Create functions with no names (closures).
- **Built-in Functions:** Use PHP's pre-defined functions (such as strlen() and date()).
- **Recursive Functions:** The functions are those that call themselves (for example, factorial calculation).
- **Function Overloading:** func_get_args() accepts multiple parameters.
- **Passing by Reference:** Use & to modify variables directly.
- **Type declarations:** Enforce the argument and return types (int, string).
- **Error Handling:** Manage errors with try, catch, and exceptions.

Function Parameters

Function Parameters in PHP

A function in PHP may be defined to accept one or more parameters. Function parameters are a comma-separated list of expressions inside the parenthesis in front of the function name while defining a function. A parameter may be of any scalar type (number, string or Boolean), an array, an object, or even another function.

Syntax

Here is the syntax for defining parameters in PHP functions

```
function foo($arg_1, $arg_2, $arg_n) {
   statements;
   return $retval;
}
```

The arguments act as the variables to be processed inside the function body. Hence, they follow the same naming conventions as any normal variable, i.e., they should start with "$" and can contain alphabets, digits and underscore.

Note There is no restriction on how many parameters can be defined.

Functions Call with Parameters

When a parameterized function needs to be called, you have to make sure that the same number of values as in the number of arguments in function's definition, are passed to it.

```
foo(val1, val2, val_n);
```

A function defined with parameters can produce a result that changes dynamically depending on the passed values.

Default Parameters in PHP

Here is the way you can set default values for function parameters in PHP so that arguments become optional.

```
<?php
   function greet($name = "Guest") {
      echo "Hello, $name!";
   }
   greet();
   echo "\n";
   greet("Rashmi");
?>
```

Output

It will produce the below result

```
Hello, Guest!
Hello, Rashmi!
```

Variable Length Arguments in PHP

In PHP, you can also define variable-length parameters if you do not know the number of parameters in advanced. Here is the way you can create functions that accept a dynamic number of arguments using '....'.

```
<?php
   function sumAll(...$numbers) {
      return array_sum($numbers);
   }
   echo sumAll(1, 2, 3, 4);
?>
```

Output

It will generate the below result

```
10
```

Function to Add Two Numbers

The following code contains the definition of addition() function with two parameters, and displays the addition of the two. The run-time output depends on the two values passed to the function.

```
<?php
   function addition($first, $second) {
      $result = $first+$second;
      echo "First number: $first \n";
      echo "Second number: $second \n";
      echo "Addition: $result";
   }

   addition(10, 20);

   $x=100;
   $y=200;
   addition($x, $y);
?>
```

Output

It will generate the following output

```
First number: 10
Second number: 20
```

```
Addition: 30
First number: 100
Second number: 200
Addition: 300
```

Formal and Actual Arguments

Sometimes the term **argument** is used for **parameter**. Actually, the two terms have a certain difference.

- A parameter refers to the variable used in function's definition, whereas an argument refers to the value passed to the function while calling.
- An argument may be a literal, a variable or an expression
- The parameters in a function definition are also often called as **formal arguments**, and what is passed is called **actual arguments**.
- The names of formal arguments and actual arguments need not be same. The value of the actual argument is assigned to the corresponding formal argument, from left to right order.
- The number of formal arguments defined in the function and the number of actual arguments passed should be same.

Example

PHP raises an **ArgumentCountError** when the number of actual arguments is less than formal arguments. However, the additional actual arguments are ignored if they are more than the formal arguments.

```
<?php
   function addition($first, $second) {
      $result = $first+$second;
      echo "First number: $first \n";
      echo "Second number: $second \n";
      echo "Addition: $result \n";
   }

   # Actual arguments more than formal arguments
   addition(10, 20, 30);

   # Actual arguments fewer than formal arguments
   $x=10;
   $y=20;
   addition($x);
?>
```

Output

It will produce the below output

```
First number: 10
Second number: 20
```

```
Addition: 30
PHP Fatal error:  Uncaught ArgumentCountError: Too few arguments
to function addition(), 1 passed in /home/cg/root/20048/main.php
on line 16 and exactly 2 expected in /home/cg/root/20048/main.php:2
```

Arguments Type Mismatch

PHP is a dynamically typed language, hence it doesn't enforce type checking when copying the value of an actual argument with a formal argument. However, if any statement inside the function body tries to perform an operation specific to a particular data type which doesn't support it, PHP raises an exception.

In the addition() function above, it is assumed that numeric arguments are passed. PHP doesn't have any objection if string arguments are passed, but the statement performing the addition encounters exception because the "+" operation is not defined for string type.

Type Error with String Arguments

Take a look at the following example

```
<?php
   function addition($first, $second) {
      $result = $first+$second;
      echo "First number: $first \n";
      echo "Second number: $second \n";
      echo "Addition: $result";
   }

   # Actual arguments are strings
   $x="Hello";
   $y="World";
   addition($x, $y);
?>
```

Output

It will create the following outcome

```
PHP Fatal error:  Uncaught TypeError: Unsupported operand types: string + string in
hello.php:5
```

However, PHP is a weakly typed language. It attempts to cast the variables into compatible type as far as possible. Hence, if one of the values passed is a string representation of a number and the second is a numeric variable, then PHP casts the string variable to numeric in order to perform the addition operation.

Automatic Type Conversion

Take a look at the following example

```
<?php
   function addition($first, $second) {
```

```
      $result = $first+$second;
      echo "First number: $first \n";
      echo "Second number: $second \n";
      echo "Addition: $result";
   }

   # Actual arguments are strings
   $x="10";
   $y=20;
   addition($x, $y);
?>
```

Output

It will produce the following output

```
First number: 10
Second number: 20
Addition: 30
```

Default Arguments

Like most of the languages that support imperative programming, a function in PHP may have one or more arguments that have a default value. As a result, such a function may be called without passing any value to it. If there is no value meant to be passed, the function will take its default value for processing. If the function call does provide a value, the default value will be overridden.

Function Definition with Default Values

Here is the way you can define a function with default values

```
function fun($arg1 = val1, $arg2 = val2) {
   Statements;
}
```

Different Ways to Call the Function

The above function can be called in different ways

```
fun();                  # Function will use defaults for both arguments
fun($x);                # Function passes $x to arg1 and uses default for arg2
fun($x, $y);    # Both arguments use the values passed
```

Example 1

Here we define a function called **greeting()** with two arguments, both having **string** as their default values. We call it by passing one string, two strings and without any argument.

```
<?php
   function  greeting($arg1="Hello", $arg2="world") {
      echo $arg1 . " ". $arg2 . PHP_EOL;
   }

   greeting();
   greeting("Thank you");
   greeting("Welcome", "back");
   greeting("PHP");
?>
```

Output

It will produce the following output

```
Hello world
Thank you world
Welcome back
PHP world
```

Example 2

You can define a function with only some of the arguments with default value, and the others to which the value must be passed.

```
<?php
   function  greeting($arg1, $arg2="World") {
      echo $arg1 . " ". $arg2 . PHP_EOL;
   }

   # greeting(); ## This will raise ArgumentCountError
   greeting("Thank you");
   greeting("Welcome", "back");
?>
```

Output

It will produce the below result

```
Thank you World
Welcome back
```

The first call (without argument) raises **ArgumentCountError** because you must pass value for the first argument. If only one value is passed, it will be used by the first argument in the list.

However, if you declare arguments with **default** before arguments without defaults, such function can be only called if values for both are passed. You cannot have a situation where the first argument uses the default, and the second using the passed value.

The **greeting()** function now has **$arg1** with default and **$arg2** without any default value.

```
function  greeting($arg1="Hello", $arg2) {
   echo $arg1 . " ". $arg2 . PHP_EOL;
}
```

If you pass a string "PHP"

```
greeting("PHP");
```

with the intension to print the result as "Hello PHP", the following error message will be displayed.

```
PHP Fatal error: Uncaught ArgumentCountError: Too few arguments to function
greeting(), 1 passed in hello.php on line 10 and exactly 2 expected
```

Example 3

Let's define a function **percent()** that calculates the percentage of marks in three subjects.

Assuming that the marks in each subject are out of 100, the **$total** argument in the function definition is given a default value as 300.

```
<?php
   function  percent($p, $c, $m, $ttl=300) {
      $per = ($p+$c+$m)*100/$ttl;
      echo "Marks obtained: \n";
      echo "Physics = $p Chemistry = $c Maths = $m \n";
      echo "Percentage = $per \n";
   }
   percent(50, 60, 70);
?>
```

Output

It will generate the following result

```
Marks obtained:
Physics = 50 Chemistry = 60 Maths = 70
Percentage = 60
```

However, if the maximum marks in each subject is 50, then you must pass the fourth value to the function, otherwise the percentage will be calculated out of 300 instead of 150.

```
<?php
   function  percent($p, $c, $m, $ttl=300) {
      $per = ($p+$c+$m)*100/$ttl;
      echo "Marks obtained: \n";
      echo "Physics = $p Chemistry = $c Maths = $m \n";
      echo "Percentage = $per \n";
   }
   percent(30, 35, 40, 150);
?>
```

Output

It will produce the following output

```
Marks obtained:
Physics = 30 Chemistry = 35 Maths = 40
Percentage = 70
```